

Implementation of Tic-Tac-Toe with AI using Minimax Algorithm

Prashanth Nayak

September 16, 2026

1 Lab Overview

Tic-Tac-Toe is a classic strategy game that serves as an excellent platform for demonstrating fundamental artificial intelligence concepts. This project implements a complete Tic-Tac-Toe game with an AI opponent that uses the Minimax algorithm to make optimal moves. The implementation showcases how AI techniques can be applied to solve seemingly simple games that nevertheless involve complex decision-making processes.

2 Theoretical Background

2.1 Minimax Algorithm

The Minimax algorithm is a decision-making algorithm used in two-player games. It assumes that both players play optimally and aims to minimize the possible loss for the worst-case scenario. The algorithm recursively evaluates all possible moves, building a game tree where:

- MAX player (AI) aims to maximize the score
- MIN player (human) aims to minimize the score
- Terminal states have values: +1 (AI wins), -1 (human wins), 0 (draw)

3 Implementation

The implementation consists of a single Python class `TicTacToe` with the following components:

- Board representation using a NumPy array
- Game state management
- Move validation logic
- Win condition checking

- Minimax algorithm implementation
- Text-based user interface

3.1 Code Implementation

```

1 import numpy as np
2 import random
3
4 class TicTacToe:
5     def __init__(self):
6         self.board = np.array([' ' * 9]) # 3x3 board represented as 1D
7         self.current_player = 'X' # Human is X, AI is O
8         self.game_over = False
9         self.winner = None
10
11     def print_board(self):
12         """Display the current board state"""
13         print("\n")
14         for i in range(3):
15             print(f" {self.board[i*3]} | {self.board[i*3+1]} | {self.board[i*3+2]} ")
16             if i < 2:
17                 print("-----")
18             print("\n")
19
20     def available_moves(self):
21         """Return list of available moves (indices of empty cells)"""
22         return [i for i, spot in enumerate(self.board) if spot == ' ']
23
24     def make_move(self, position, player):
25         """Make a move on the board"""
26         if self.board[position] == ' ':
27             self.board[position] = player
28             return True
29         return False
30
31     def check_winner(self):
32         """Check if there's a winner or if the game is a draw"""
33         # Check rows
34         for i in range(0, 9, 3):
35             if self.board[i] != ' ' and self.board[i] == self.board[i+1]
36             == self.board[i+2]:
37                 return self.board[i]
38
39         # Check columns
40         for i in range(3):
41             if self.board[i] != ' ' and self.board[i] == self.board[i+3]
42             == self.board[i+6]:
43                 return self.board[i]
44
45         # Check diagonals

```

```

44         if self.board[0] != ' ' and self.board[0] == self.board[4] == self
45         .board[8]:
46             return self.board[0]
47         if self.board[2] != ' ' and self.board[2] == self.board[4] == self
48         .board[6]:
49             return self.board[2]
50
51         # Check for draw
52         if ' ' not in self.board:
53             return 'Draw'
54
55         return None
56
57     def minimax(self, board, depth, is_maximizing):
58         """Minimax algorithm for AI move selection"""
59         # Base cases: check for terminal states
60         result = self.check_winner()
61         if result == 'O': # AI wins
62             return 1
63         elif result == 'X': # Human wins
64             return -1
65         elif result == 'Draw':
66             return 0
67
68         if is_maximizing:
69             best_score = -float('inf')
70             for move in self.available_moves():
71                 board[move] = 'O'
72                 score = self.minimax(board, depth + 1, False)
73                 board[move] = ' '
74                 best_score = max(score, best_score)
75             return best_score
76         else:
77             best_score = float('inf')
78             for move in self.available_moves():
79                 board[move] = 'X'
80                 score = self.minimax(board, depth + 1, True)
81                 board[move] = ' '
82                 best_score = min(score, best_score)
83             return best_score
84
85     def ai_move(self):
86         """Determine the best move for the AI using minimax"""
87         best_score = -float('inf')
88         best_move = None
89
90         for move in self.available_moves():
91             self.board[move] = 'O'
92             score = self.minimax(self.board, 0, False)
93             self.board[move] = ' '
94
95             if score > best_score:
96                 best_score = score
97                 best_move = move

```

```

96         return best_move
97
98
99     def play(self):
100         """Main game loop"""
101         print("Welcome to Tic-Tac-Toe!")
102         print("You are X, and the AI is O.")
103         print("Enter a number (0-8) to make your move:")
104         print(" 0 | 1 | 2 ")
105         print("-----")
106         print(" 3 | 4 | 5 ")
107         print("-----")
108         print(" 6 | 7 | 8 ")
109
110         while not self.game_over:
111             self.print_board()
112
113             if self.current_player == 'X':
114                 # Human's turn
115                 try:
116                     move = int(input("Your move (0-8): "))
117                     if move < 0 or move > 8:
118                         print("Please enter a number between 0 and 8.")
119                         continue
120                     if not self.make_move(move, 'X'):
121                         print("That position is already taken. Try again.")
122                     continue
123                 except ValueError:
124                     print("Please enter a valid number.")
125                     continue
126             else:
127                 # AI's turn
128                 print("AI is thinking...")
129                 move = self.ai_move()
130                 self.make_move(move, 'O')
131                 print(f"AI chooses position {move}")
132
133             # Check for game over
134             result = self.check_winner()
135             if result:
136                 self.game_over = True
137                 self.winner = result
138                 self.print_board()
139                 if result == 'Draw':
140                     print("It's a draw!")
141                 else:
142                     print(f"{result} wins!")
143             else:
144                 # Switch players
145                 self.current_player = 'O' if self.current_player == 'X'
146                 else 'X'
147
148 # Run the game

```

```

148 if __name__ == "__main__":
149     game = TicTacToe()
150     game.play()

```

Listing 1: Complete Tic-Tac-Toe AI Implementation

4 Instructions for Use

4.1 Prerequisites

- Python 3.x installed on your system
- NumPy library (install with `pip install numpy`)

4.2 Running the Game

1. Create a new Python file named `tic_tac_toe_ai.py`
2. Copy the code from Listing 1 into the file
3. Save the file
4. Open a terminal and navigate to the directory containing the file
5. Run the game with: `python tic_tac_toe_ai.py`

4.3 Gameplay Instructions

- You play as 'X' and the AI is 'O'
- When it's your turn, enter a number from 0 to 8 corresponding to the position on the board:

0	1	2
3	4	5
6	7	8

- The AI will automatically make its move after yours
- The game continues until someone wins or it's a draw

5 Conclusion

This implementation of Tic-Tac-Toe with an AI opponent provides a practical demonstration of fundamental AI concepts. The Minimax algorithm, while simple in concept, forms the basis for more complex game-playing algorithms used in advanced AI systems. This project serves as an excellent foundation for understanding adversarial search and decision-making in AI.

References

1. Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
2. Rich, E., Knight, K., & Nair, S. B. (2009). Artificial Intelligence. McGraw-Hill.
3. Nilsson, N. J. (2014). Principles of Artificial Intelligence. Morgan Kaufmann.